Session 16 (AIML) - Assignment Questions

Name**: Vaishnavi Praveen Rathi**
College**: Zeal Polytechnic, Narhe, Pune**
Branch**: Artificial Intelligence and Machine Learning (AIML)**
Domain**: AIML (Artificial Intelligence and Machine Learning)**

Github Repo Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session16/Assignment16.ipynb

Dataset Used: – Insurance.csv (Already Provided)
Availaible on Github File Link.

Q1. Load Dataset

Output Screenshots:

```
First 10 Rows of the Dataset:
   age     sex   bmi  children smoker     region  expenses
0   19  female  27.9         0    yes  southwest  16884.92
1   18    male  33.8         1     no  southeast   1725.55
2   28    male  33.0         3     no  southeast   4449.46
3   33    male  22.7         0     no  northwest  21984.47
4   32    male  28.9         0     no  northwest   3866.86
5   31  female  25.7         0     no  southeast   3756.62
6   46  female  33.4         1     no  southeast   8240.59
7   37  female  27.7         3     no  northwest   7281.51
8   37    male  29.8         2     no  northeast   6406.41
9   60  female  25.8         0     no  northwest  28923.14
```

Program Code:

```
[10]: '''
      Q1. Load the insurance dataset using pandas and display
      the first 10 rows of the data.
      '''

      import pandas as pd

      # Load the dataset
      df = pd.read_csv("insurance.csv")

      # Display first 10 rows
      print("First 10 Rows of the Dataset:")
      print(df.head(10))
```

Q2. Dataset Information

Output Screenshot:

```
Shape of the Dataset:
(1338, 7)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   expenses  1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB

Statistical Summary:
                age          bmi     children      expenses
count  1338.000000  1338.000000  1338.000000   1338.000000
mean     39.207025    30.665471     1.094918  13270.422414
std      14.049960     6.098382     1.205493  12110.011240
min      18.000000    16.000000     0.000000   1121.870000
25%      27.000000    26.300000     0.000000   4740.287500
50%      39.000000    30.400000     1.000000   9382.030000
75%      51.000000    34.700000     2.000000  16639.915000
max      64.000000    53.100000     5.000000  63770.430000
```

Program Code:

[11]:
```
'''
Q2. Use pandas functions to get basic information about the dataset:

- Shape of the data (shape)
- Column names and data types (info())
- Statistical summary of numerical columns (describe())
'''

import pandas as pd

# Load the dataset
df = pd.read_csv("insurance.csv")

# Shape of the dataset
print("Shape of the Dataset:")
print(df.shape)

# Column names and data types
print("\nDataset Information:")
df.info()

# Statistical summary
print("\nStatistical Summary:")
print(df.describe())
```

Q3.Missing Values

Output Screenshot:

```
Null Values in Each Column:
age         0
sex         0
bmi         0
children    0
smoker      0
region      0
expenses    0
dtype: int64
```

Program Code:

[12]:
```
'''
Q3. Check if the dataset contains any null/missing values.

Show the count of null values for each column.
'''

import pandas as pd

# Load the dataset
df = pd.read_csv("insurance.csv")

# Count null values
print("Null Values in Each Column:")
print(df.isnull().sum())
```

Q4. Column Types

Output Screenshot:

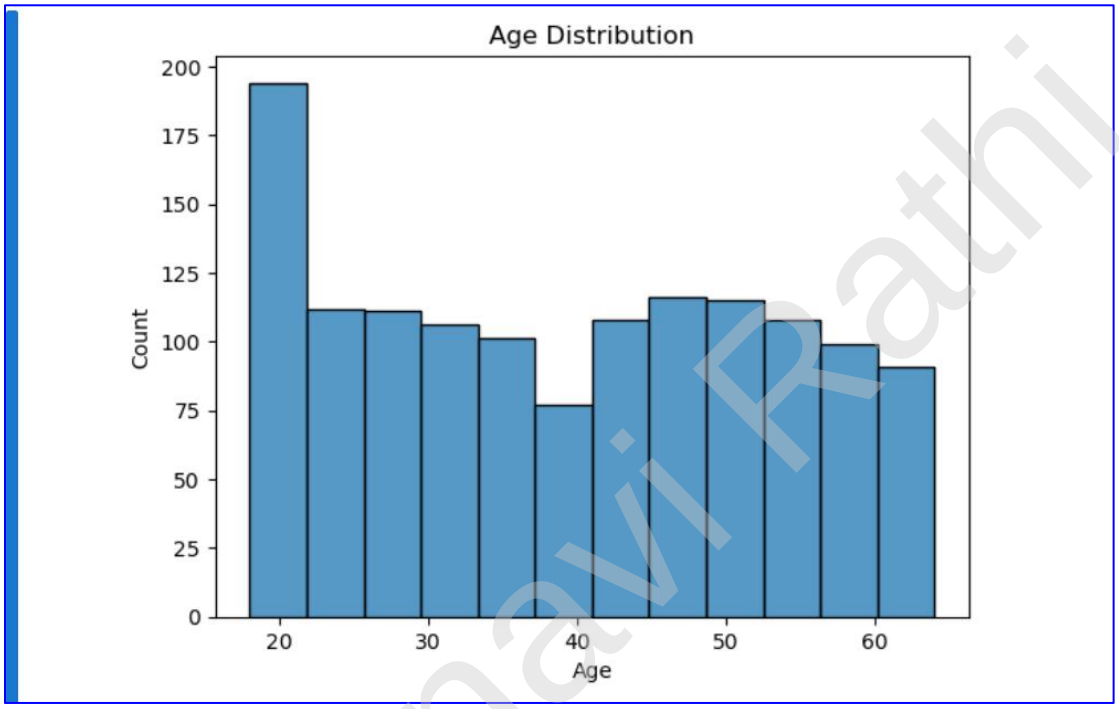
```
Numerical Columns:
Index(['age', 'bmi', 'children', 'expenses'], dtype='object')

Categorical Columns:
Index(['sex', 'smoker', 'region'], dtype='object')
```

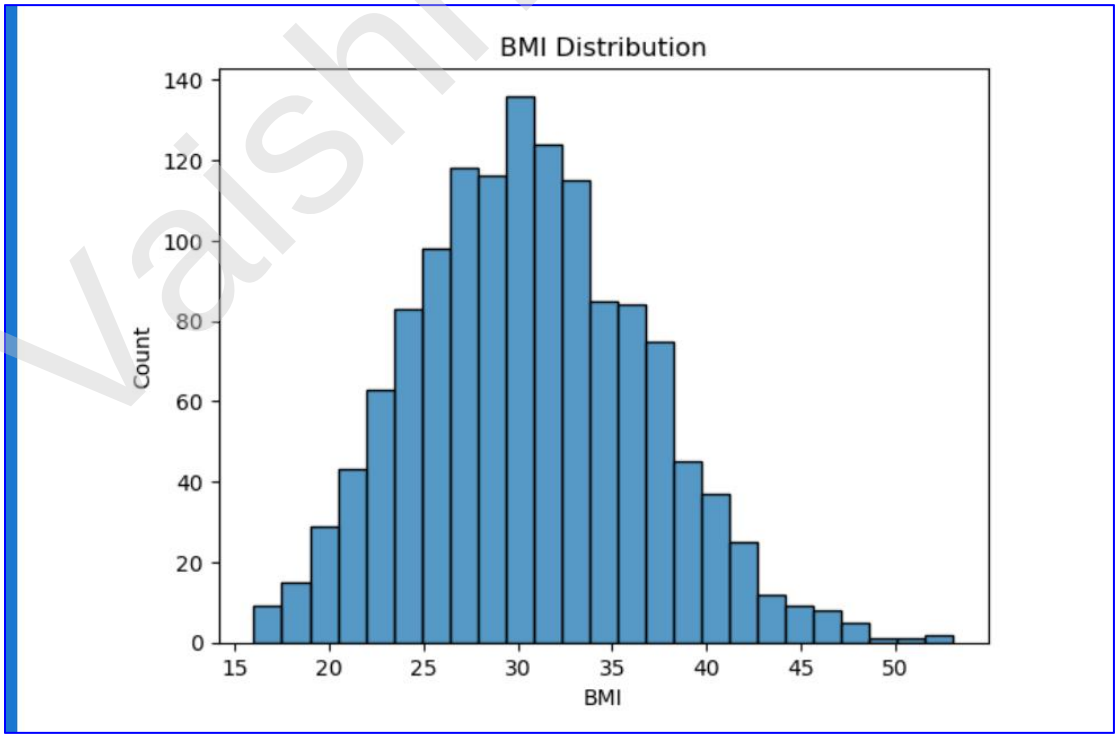
Program Code:

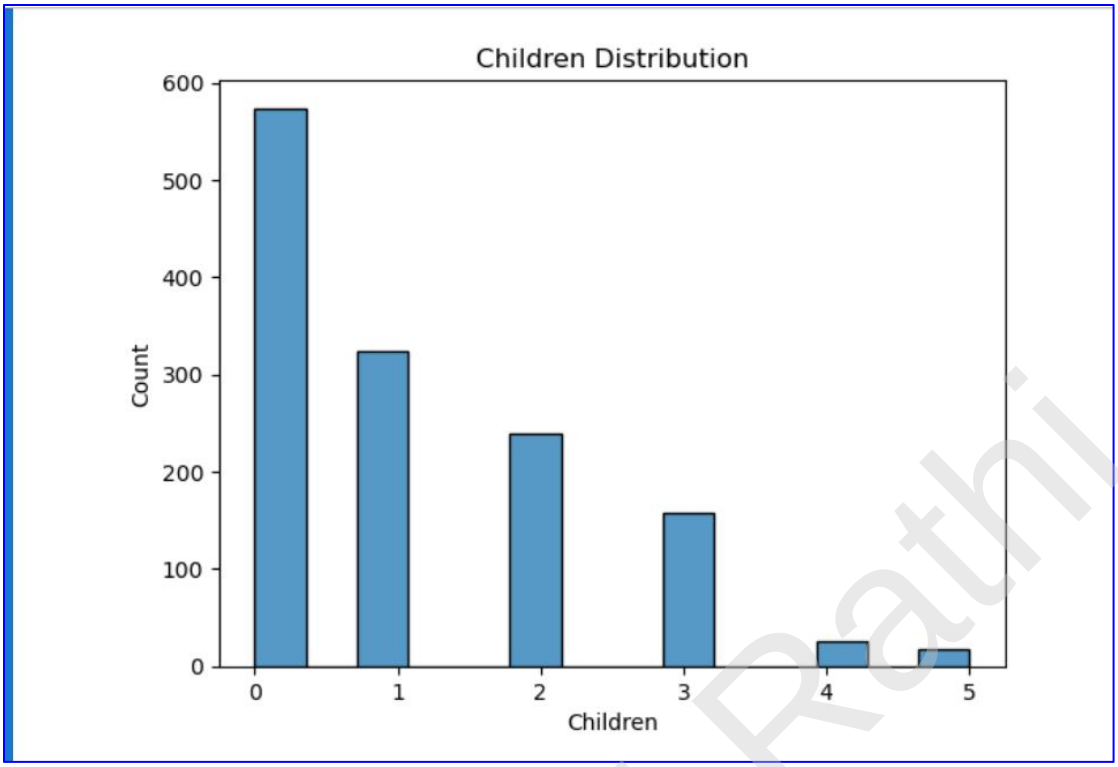
[13]:
```
'''
Q4. Identify numerical and categorical columns in the dataset.

Print them separately.
'''

import pandas as pd

# Load the dataset
df = pd.read_csv("insurance.csv")

# Identify numerical columns
numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns

# Identify categorical columns
categorical_columns = df.select_dtypes(include=['object']).columns

# Print numerical columns
print("Numerical Columns:")
print(numerical_columns)

# Print categorical columns
print("\nCategorical Columns:")
print(categorical_columns)
```

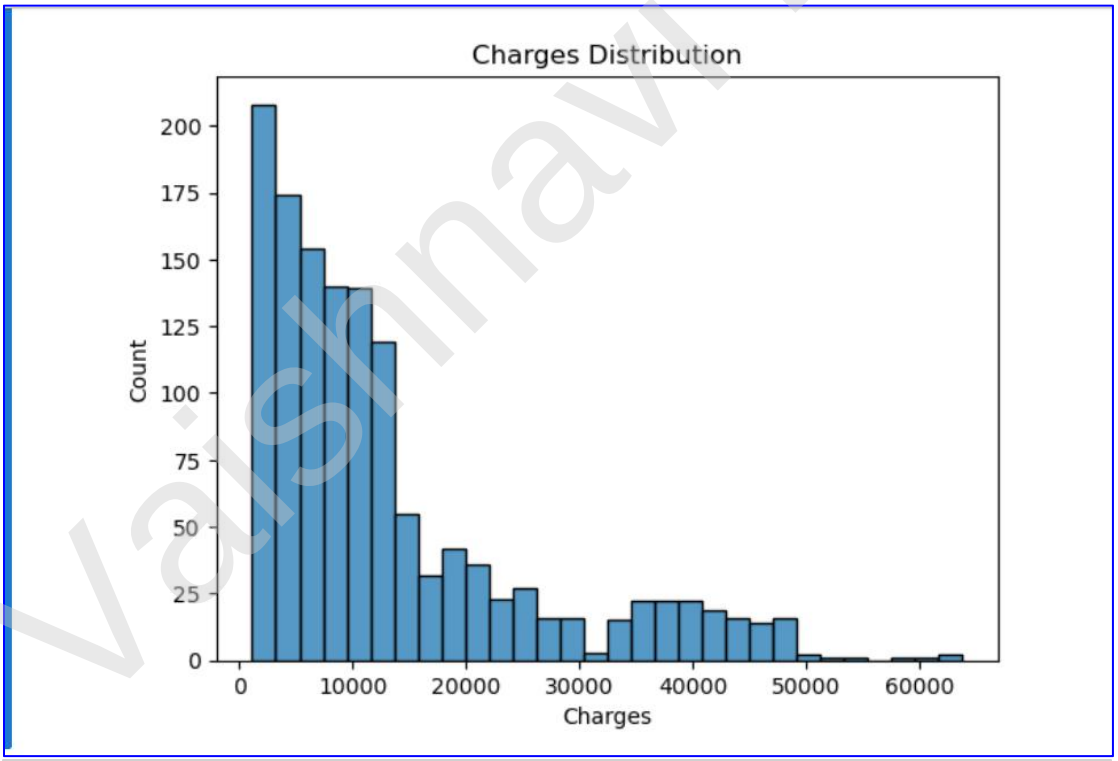
Q5. Distribution Plots

Output Screenshot:









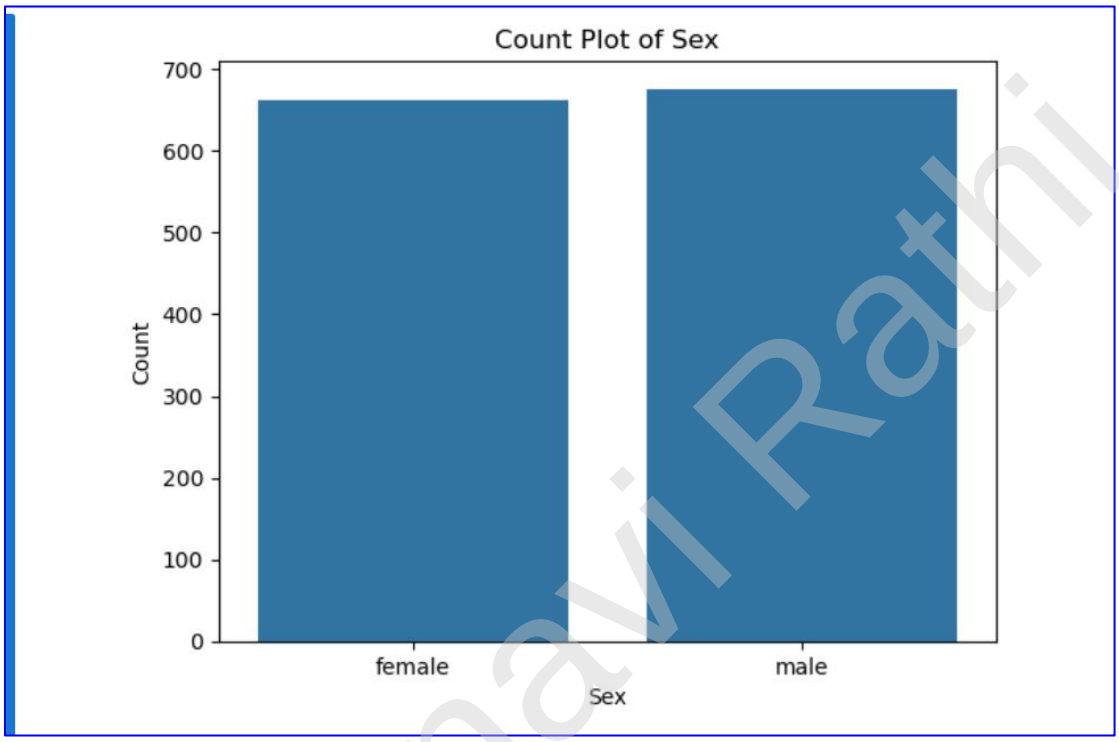
Program Code:

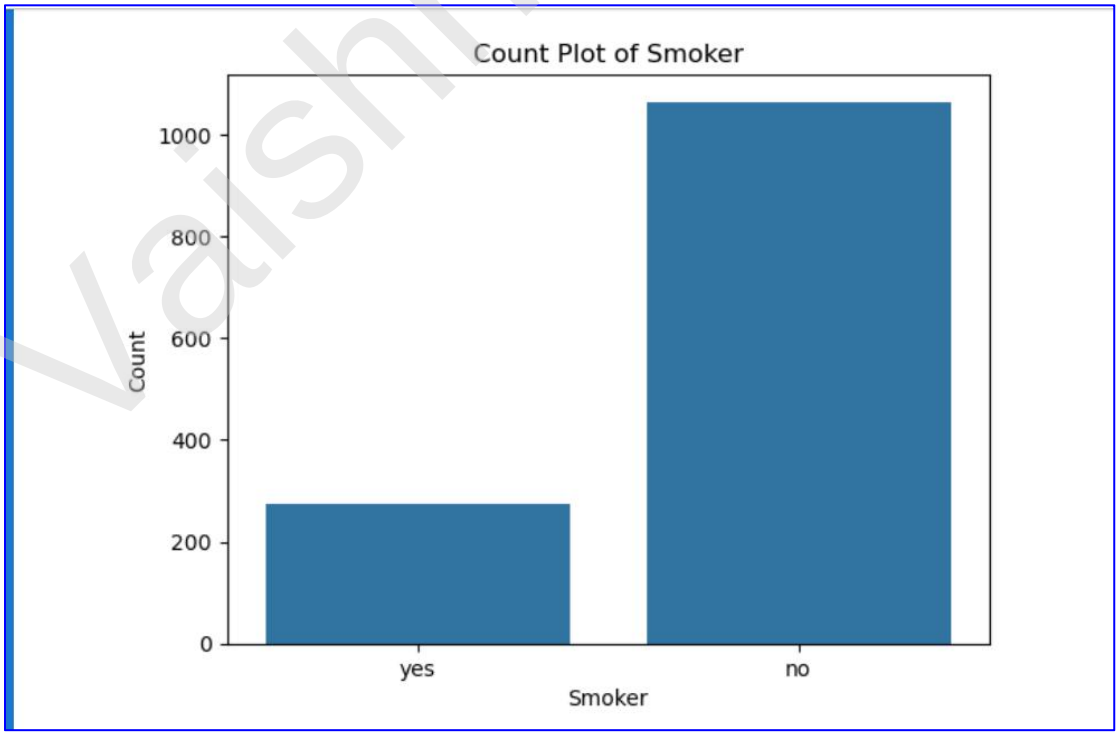
[17]:
```
'''
Q5. Create distribution plots for all numerical columns
(age, bmi, children, charges)

- Use sns.histplot() or sns.distplot()
- Create one plot per column (total 4 plots)
- Add meaningful titles and labels
'''

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("insurance.csv")

# Plot for Age
sns.histplot(df["age"])
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()

# Plot for BMI
sns.histplot(df["bmi"])
plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.ylabel("Count")
plt.show()
```

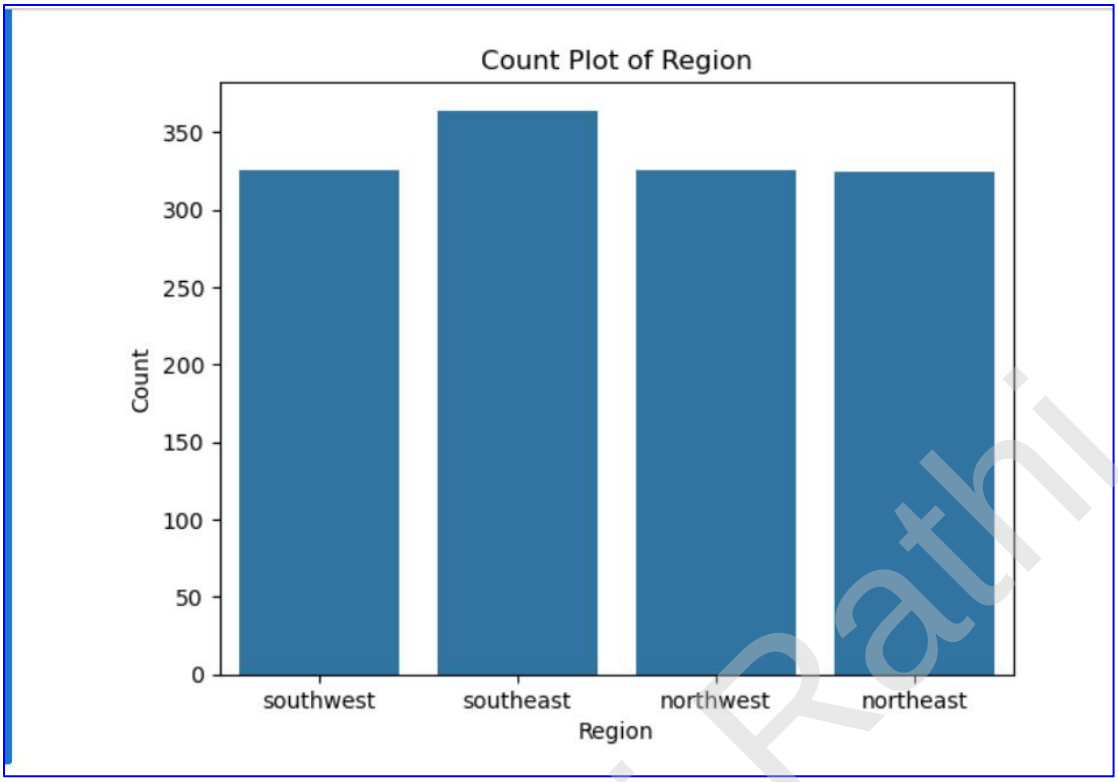
```
# Plot for Children
sns.histplot(df["children"])
plt.title("Children Distribution")
plt.xlabel("Children")
plt.ylabel("Count")
plt.show()

# Plot for Charges
sns.histplot(df["expenses"])
plt.title("Charges Distribution")
plt.xlabel("Charges")
plt.ylabel("Count")
plt.show()
```

Q6. Count Plots

Output Screenshot:







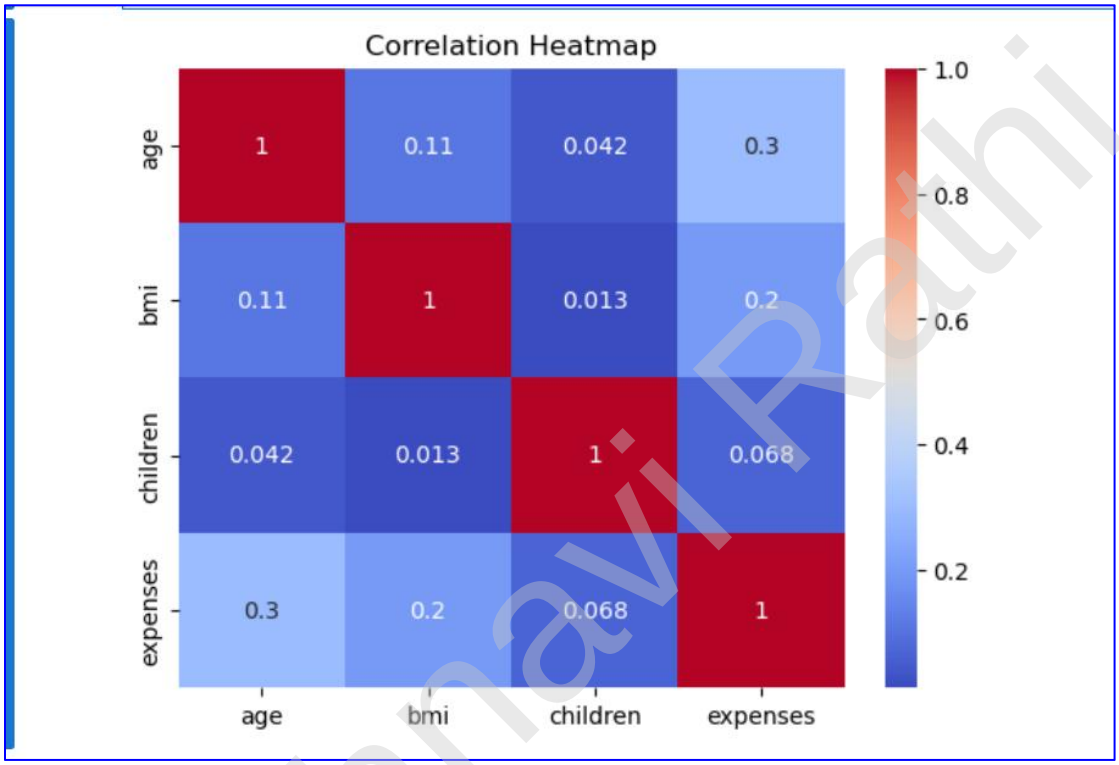
Program Code:

```
[18]: '''
Q6. Create count plots for all categorical columns (sex, smoker, region)

- Use sns.countplot() for each column
- Add titles and labels for clarity
'''

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("insurance.csv")

# Count plot for Sex
sns.countplot(x="sex", data=df)
plt.title("Count Plot of Sex")
plt.xlabel("Sex")
plt.ylabel("Count")
plt.show()

# Count plot for Smoker
sns.countplot(x="smoker", data=df)
plt.title("Count Plot of Smoker")
plt.xlabel("Smoker")
plt.ylabel("Count")
plt.show()
```

```
# Count plot for Region
sns.countplot(x="region", data=df)
plt.title("Count Plot of Region")
plt.xlabel("Region")
plt.ylabel("Count")
plt.show()
```

Q7. Correlation Heatmap

Output Screenshot:



Program Code:

[19]:
```
'''
Q7. Create a heatmap to show the correlation between numerical variables.

- Use sns.heatmap()
- Annotate the values
- Use a suitable color palette
'''

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("insurance.csv")

# Calculate correlation
corr = df.corr(numeric_only=True)

# Create heatmap
sns.heatmap(corr, annot=True, cmap="coolwarm")

# Add title
plt.title("Correlation Heatmap")
plt.show()
```

Q8.Charges Analysis

Output Screenshot:

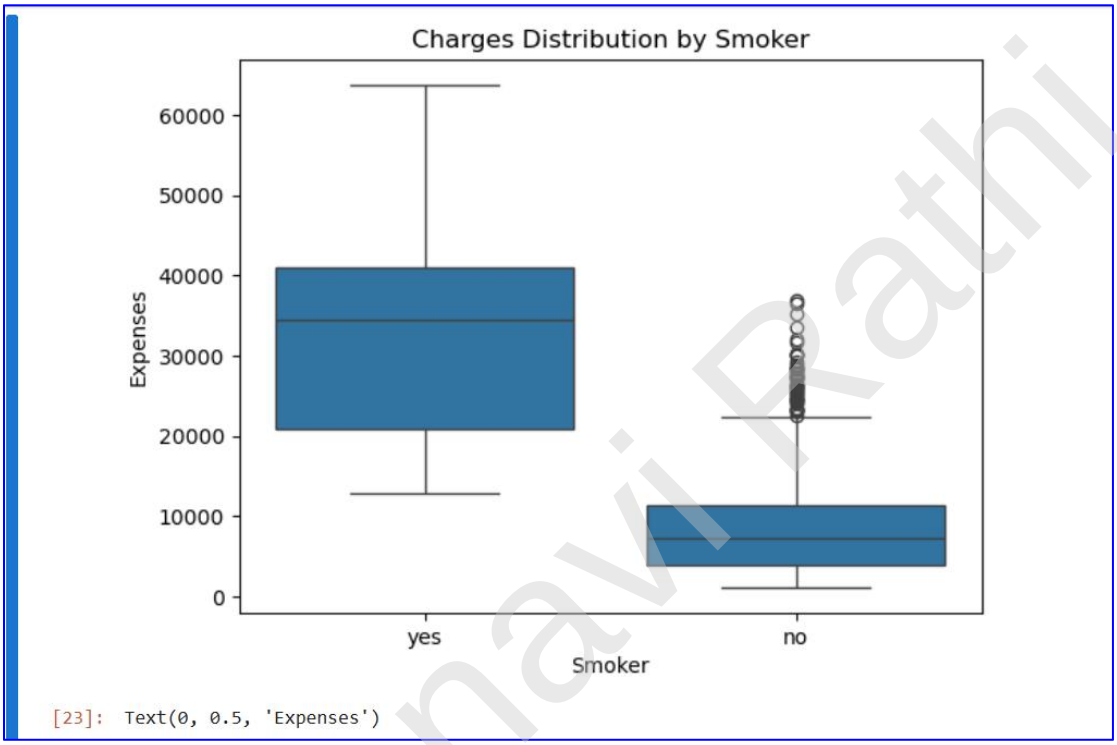
```
Average Insurance Charges:
13270.422414050823

Maximum Insurance Charges:
63770.43

Minimum Insurance Charges:
1121.87

Average Insurance Charges for Smokers and Non-Smokers:
smoker
no      8434.268449
yes    32050.231971
Name: expenses, dtype: float64
```

Program Code:

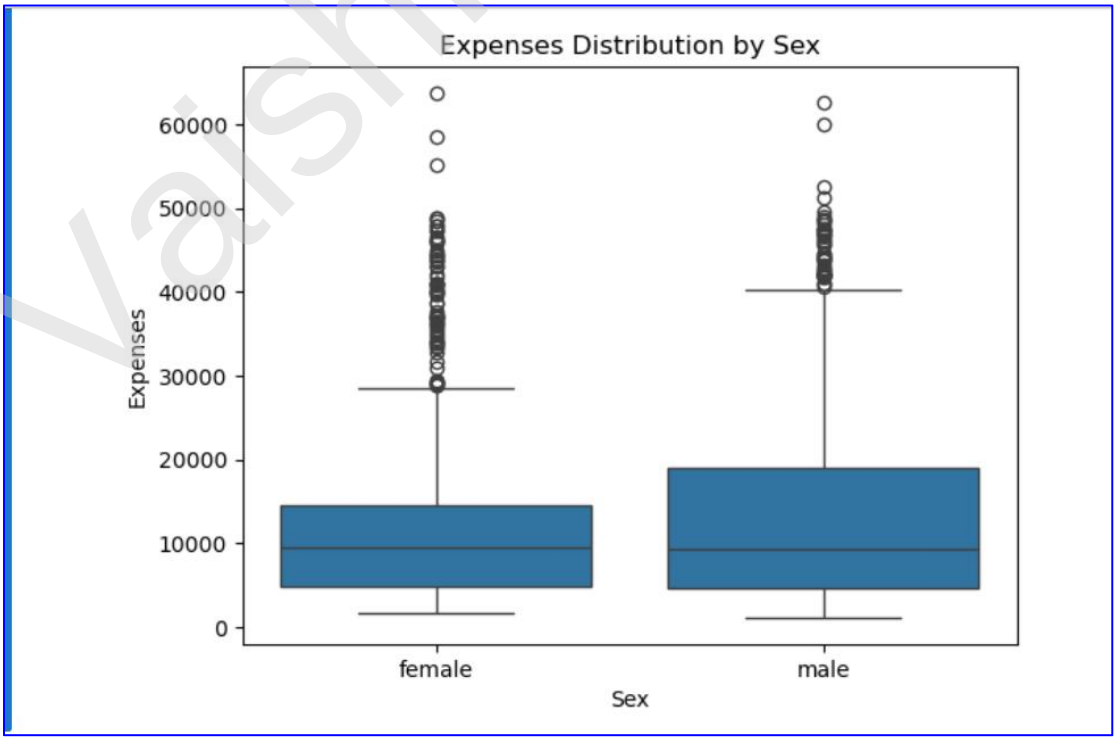
```
[20]: '''
Q8. Perform the following analysis on the charges column:

- Find the average insurance charges
- Find the maximum and minimum charges
- Group by smoker and compare the average charges for smokers vs non-smokers
'''

import pandas as pd

# Load the dataset
df = pd.read_csv("insurance.csv")

# Find the average insurance charges
print("Average Insurance Charges:")
print(df["expenses"].mean())

# Find the maximum and minimum charges
print("\nMaximum Insurance Charges:")
print(df["expenses"].max())

print("\nMinimum Insurance Charges:")
print(df["expenses"].min())

# Group by smoker and compare the average charges
print("\nAverage Insurance Charges for Smokers and Non-Smokers:")
print(df.groupby("smoker")["expenses"].mean())
```

Q9.Box Plots

Output Screenshot:




```
[23]: Text(0, 0.5, 'Expenses')
```



Program Code:

[23]:
```
'''
Q9. Create a boxplot to show the distribution of charges with respect to smoker
and sex.

(You can create two separate boxplots or use hue parameter)
'''

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("insurance.csv")

# Boxplot for Charges vs Smoker
sns.boxplot(x="smoker", y="expenses", data=df)
plt.title("Charges Distribution by Smoker")
plt.xlabel("Smoker")
plt.ylabel("Expenses")
plt.show()

# Boxplot for Charges vs Sex
sns.boxplot(x="sex", y="expenses", data=df)
plt.title("Expenses Distribution by Sex")
plt.xlabel("Sex")
plt.ylabel("Expenses")
```

Q10. Mini Project / Analysis Summary

Short Summary Answers :

[25]:
```
'''
Q10. (Mini Project / Analysis Summary)

1. Average Age and BMI:
   - The average age and average BMI can be obtained using the mean() function.

2. Smoking and Insurance Charges:
   - Smokers have significantly higher average insurance charges than non-smokers.

3. Region with the Highest Number of Customers:
   - The region with the highest number of customers can be identified using value_counts().

4. Observations from the Visualizations:
   - Age and BMI are distributed across a wide range of values.
   - Most customers have fewer children.
   - Smokers generally have much higher insurance charges than non-smokers.
   - The heatmap shows that charges have a strong positive correlation with smoking and
     a moderate correlation with age.
'''
```

Optional Task

Github File Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session16/IrsisEDA%20Optional%20Task.ipynb

Dataset Used: Kaggle - https://www.kaggle.com/datasets/uciml/iris

Why I Chose the Iris Dataset ?

- I used the **Iris dataset** for this **Exploratory Data Analysis (EDA)**.
- It is a simple, clean, and widely used dataset in machine learning.
- It contains numerical features and three different flower species.
- It helped me understand the dataset structure and perform data exploration.
- I used it to create visualizations, identify patterns and relationships, and practice basic EDA techniques.
- This analysis helped me understand the data before applying machine learning algorithms.

Program Output:

1. Import Libraries

```
[4]: # Import Libraries
     import pandas as pd
     import numpy as np
     import seaborn as sns
     import matplotlib.pyplot as plt
```

2. Load Dataset

```
[24]: # Load the dataset

      df = pd.read_csv("Iris.csv")
```

3. First 5 Rows

```
[6]: # Display first 5 rows
     print("First 5 Rows:")
     df.head()
```

```
First 5 Rows:
```

[6]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

4. Last 5 Rows

```
[26]: # Display last 5 rows
      print("Last 5 Rows:")
      df.tail()
```

```
Last 5 Rows:
```

[26]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

5. Dataset Shape

```
[8]: # Shape of the dataset
     print("Shape of Dataset:")
     print(df.shape)
```

```
Shape of Dataset:
(150, 6)
```

6. Column Names

```
[9]: # Display column names
     print("Column Names:")
     print(df.columns)
```

```
Column Names:
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')
```

7. Dataset Information

```
[10]: # Display dataset information
      df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

8. Data Types

```
[11]: # Display data types
      print(df.dtypes)
```

```
Id                int64
SepalLengthCm   float64
SepalWidthCm    float64
PetalLengthCm   float64
PetalWidthCm    float64
Species          object
dtype: object
```

9. Summary Statistics

```
[12]: # Display summary statistics
      df.describe()
```

[12]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000


```
[12]: # Display summary statistics
      df.describe()
```

[12]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

10. Missing Values

```
[13]: # Check missing values
      print(df.isnull().sum())
```

```
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64
```

11. Unique Values

```
[27]: # Count unique values
      print(df.nunique())
```

```
Id               150
SepalLengthCm     35
SepalWidthCm      23
PetalLengthCm     43
PetalWidthCm      22
Species            3
dtype: int64
```

12. Random Sample

```
[15]: # Display random 5 rows
      df.sample(5)
```

[15]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
67	68	5.8	2.7	4.1	1.0	Iris-versicolor
68	69	6.2	2.2	4.5	1.5	Iris-versicolor
17	18	5.1	3.5	1.4	0.3	Iris-setosa
77	78	6.7	3.0	5.0	1.7	Iris-versicolor
114	115	5.8	2.8	5.1	2.4	Iris-virginica

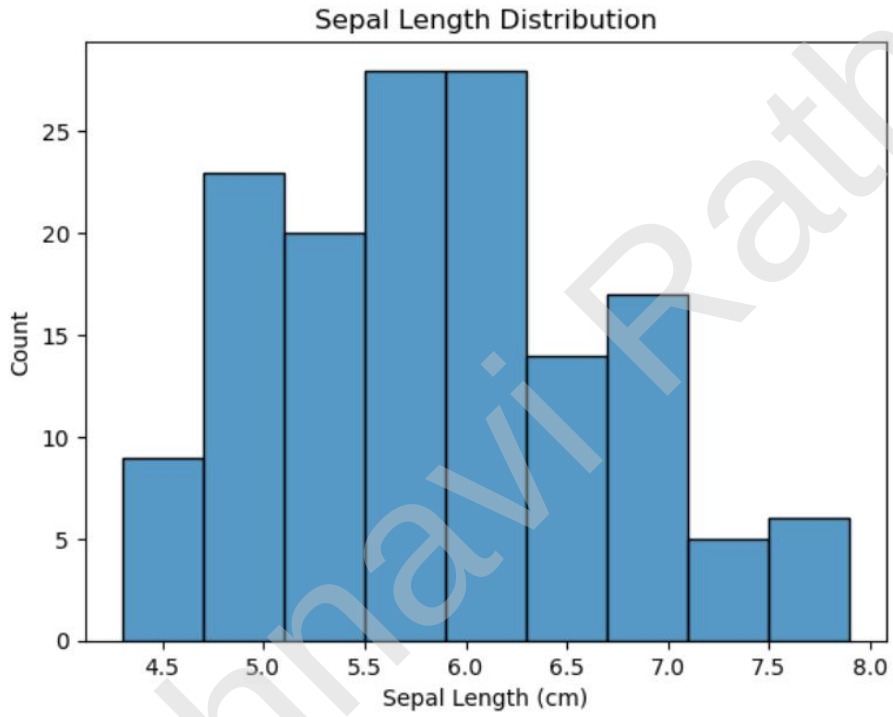
13. Species Count

```
[16]: # Count of each species
      print(df["Species"].value_counts())
```

```
Species
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

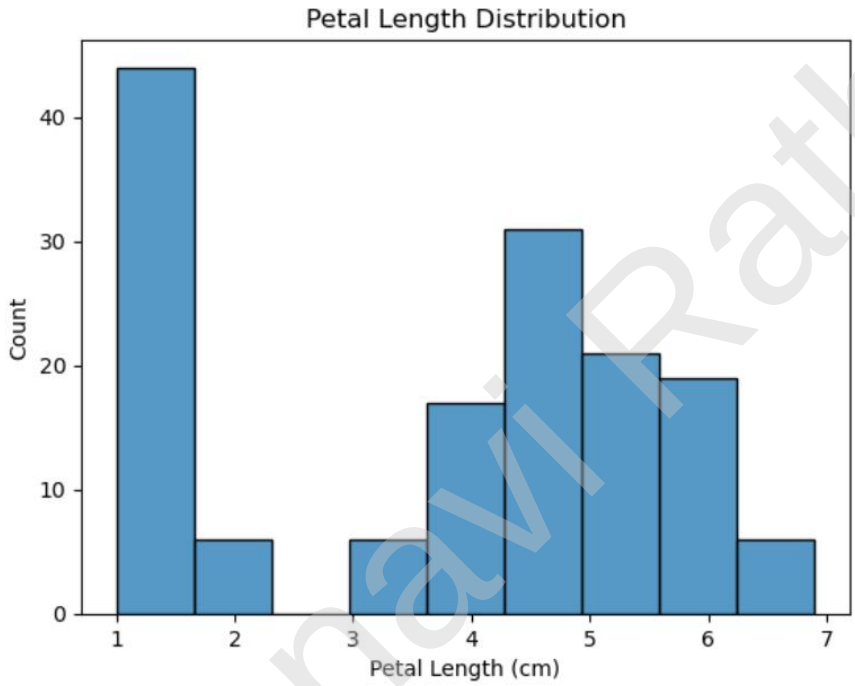
14. Sepal Distribution

```
[17]: sns.histplot(df["SepalLengthCm"])

      plt.title("Sepal Length Distribution")
      plt.xlabel("Sepal Length (cm)")
      plt.ylabel("Count")

      plt.show()
```



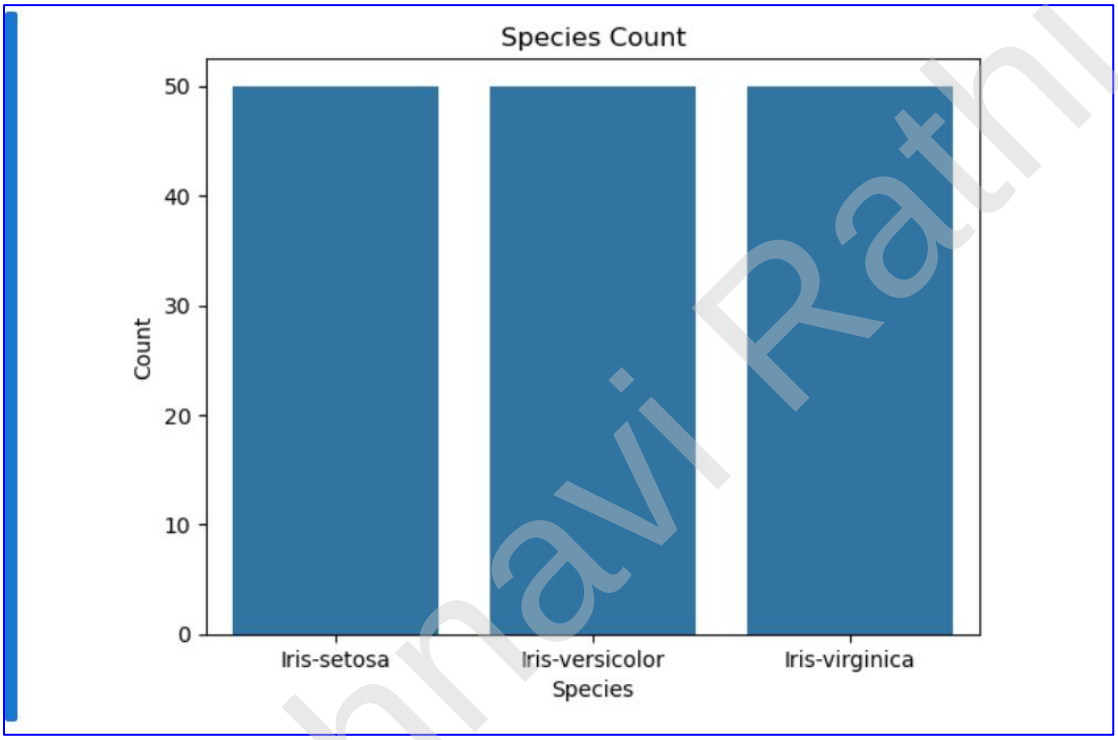
15. Petal Distribution

```
[18]: sns.histplot(df["PetalLengthCm"])

      plt.title("Petal Length Distribution")
      plt.xlabel("Petal Length (cm)")
      plt.ylabel("Count")

      plt.show()
```



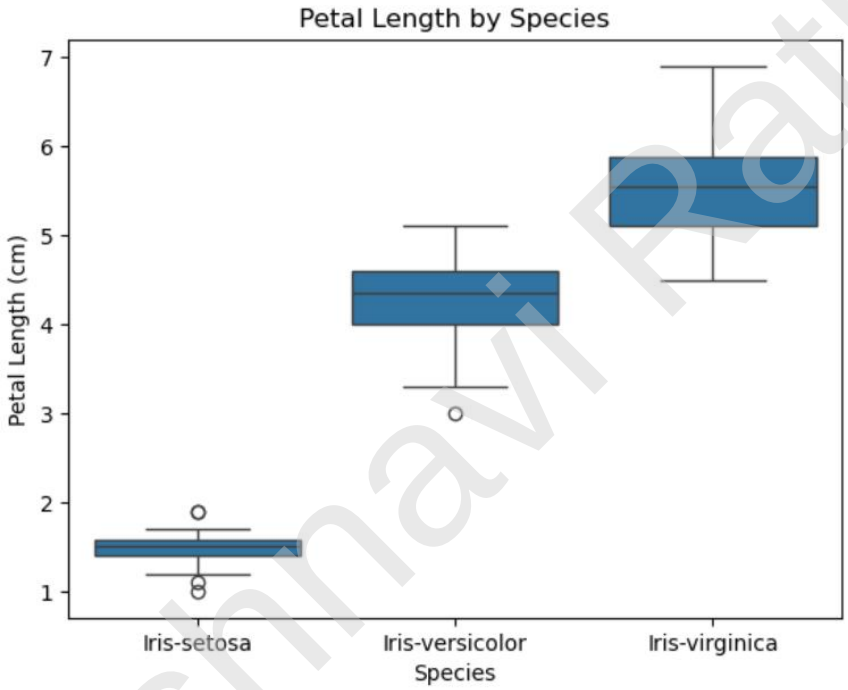
16. Species Count Plot

```
[19]: sns.countplot(x="Species", data=df)

      plt.title("Species Count")
      plt.xlabel("Species")
      plt.ylabel("Count")

      plt.show()
```



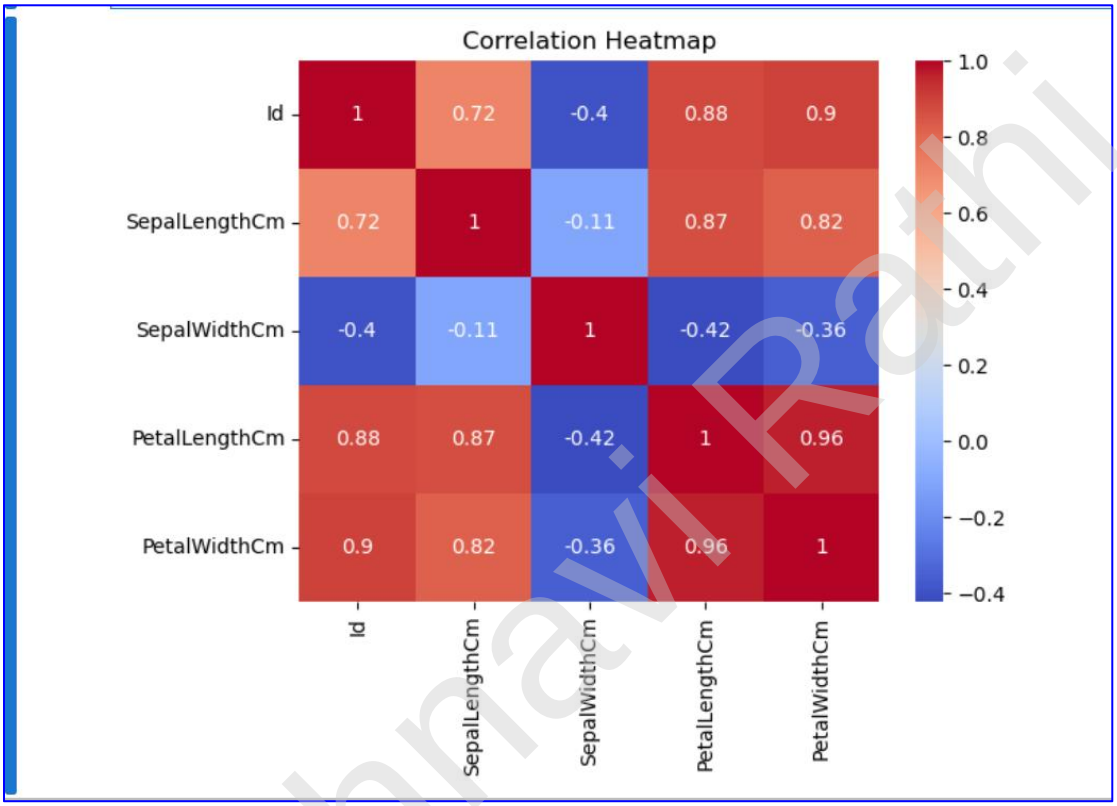
17. Species Box Plot

```
[20]: sns.boxplot(x="Species", y="PetalLengthCm", data=df)

      plt.title("Petal Length by Species")
      plt.xlabel("Species")
      plt.ylabel("Petal Length (cm)")

      plt.show()
```



18. Correlation Heatmap

```
[22]: corr = df.select_dtypes(include=["number"]).corr()

      sns.heatmap(corr, annot=True, cmap="coolwarm")
      plt.title("Correlation Heatmap")
      plt.show()
```



19. Average Values

```
[23]: print("Average Sepal Length:")
      print(df["SepalLengthCm"].mean())

      print("\nAverage Petal Length:")
      print(df["PetalLengthCm"].mean())
```

```
Average Sepal Length:
5.843333333333334

Average Petal Length:
3.758666666666666
```

20. Analysis Summary

Analysis Summary

• Flower Species

- The Iris dataset contains three flower species:
  - Iris-setosa
  - Iris-versicolor
  - Iris-virginica

• Data Quality

- There are no missing values in the dataset.
- The dataset is clean and suitable for analysis.

• Correlation

- Petal Length and Petal Width have a strong positive correlation.

• Observations

- Iris-setosa is clearly distinguishable from the other two species based on petal measurements.
- Iris-versicolor and Iris-virginica have more similar feature values.

• Conclusion

- The dataset is clean and suitable for Exploratory Data Analysis (EDA) and machine learning. '''